Øving 9: Minimale spenntrær

Collapse context

Du må oppnå minst 9 av totalt 23 mulige poeng for å denne øvingen godkjent. Hver flervalgsoppgave er verdt 1 poeng, mens hver kodeoppgave er verdt 3 poeng. For informasjon om øvingsopplegget, se **nettsidene**. Trenger du hjelp, så **ta kontakt**.

Løsningsforslaget publiseres på **Canvas** etter innleveringsfristen.

Question 1: Refleksjon

Dette er en refleksjonsoppgave. Denne oppgaven teller ikke mot antall oppgaver som må løses for å få godkjent øvingen. For mer informasjon om refleksjonsoppgavene, se **nettsidene**.

Refleksjonsoppgavene handler kun om refleksjon rundt forståelse av og arbeid med fagstoffet og øvingen. Pass på så du ikke inkluderer noen **sensitive personopplysninger** i refleksjonen din.

Før du begynner å gjøre øvingen, reflekter over følgende:

- Hvilke deler av pensum knyttet til denne øvingen er du usikker på?
- Hva tror du at du kommer til å lære ved å gjøre denne øvingen?

Etter du er ferdig med øvingen, reflekter over følgende:

- Hva forstår du nå, som du ikke forsto før du begynte å jobbe med øvingen?
- Hvilke nye sammenhenger ser du?
- Kan du komme på flere anvendelser av stoffet fra denne øvingen? I så fall hvilke?
- Er det noe du fortsatt er usikker på? I så fall hva?
- Hvis du skulle gjort øvingen på nytt, hva ville du gjort annerledes?

Question 2: Skog-implementasjonen av disjunkte mengder

Hvilke påstander stemmer om skog-implementasjonen av disjunkte mengder (*disjoint-set forests*)?

- [ ] Rangen $u.rank$ for en node u er en øvre grense for høyden til u.
- [ ] Rangen $u.rank$ for en node u er nøyaktig lik høyden til u.
- [ ] Etter $\textsc{Find-Set}(x)$ vil alle noder i treet som x tilhører ha samme forelder $x.p$.
- [ ] Uten stikomprimeringsheuristikken (*the path compression heuristic*) vil ikke $\textsc{Find-Set}(x)$ finne riktig representant.

Question 3: Minimale spenntrær

Hvis alle kantene i en sammenhengende graf har forskjellige vekter, hvor mange ulike minimale spenntrær kan grafen ha?

- ○ Alltid flere
- ○ Noen ganger kun ett, andre ganger flere
- ○ Alltid kun ett

Question 4: Minimale spenntrær

Hvis alle kantene i en sammenhengende graf har samme vekt, hvor mange ulike minimale spenntrær kan grafen ha?

- ○ Alltid flere
- ○ Noen ganger kun ett, andre ganger flere
- ○ Alltid kun ett

Question 5: Minimale spenntrær

Hvis en kant (u, v) har strengt lavere kantvekt enn alle de andre kantene i en sammenhengende graf, vil denne kanten være med i et minimalt spenntre?

- ○ Ja, alltid
- ○ Nei, aldri
- ○ I noen tilfeller

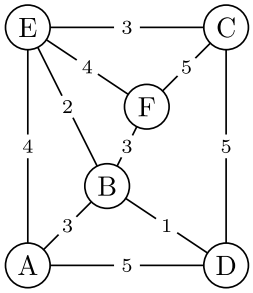
Question 6: Minimale spenntrær

Hvis en kant (u, v) har strengt høyere kantvekt enn alle de andre kantene i en sammenhengende graf G med $|\mathrm{E}| \geqslant |\mathrm{V}|$, vil denne kanten være med i et minimalt spenntre?

- ○ Nei, aldri
- ○ I noen tilfeller
- ○ Ja, alltid

Question 7: Minimale spenntrær

Hva er vekten av et minimalt spenntre i denne grafen?



- 15
- 11
- 12
- 13
- 14

Question 8: Generic-MST

I GENERIC-MST bygges et minimalt spenntre A gradvis ved at man i hver iterasjon velger en ny kant som blir lagt til i A. Hvilke egenskaper må kanten man velger ha?

- [ ] Det må finnes en oppdeling av nodene i to mengder, slik at kanten knytter sammen to noder som ikke er i samme mengde og har lavest vekt blant alle slike kanter. Det er ingen begrensing på om kantene i A knytter sammen to noder i hver sin mengde.
- [ ] Den må ha størst vekt blant alle kanter som ikke er i A.
- [ ] Den må ha lavest vekt blant alle kanter som ikke er i A.
- [ ] Det må finnes en oppdeling av nodene i to mengder, slik at kanten knytter sammen to noder som ikke er i samme mengde og har lavest vekt blant alle slike kanter. Kantene i A kan ikke knytte sammen to noder som kommer fra hver sin mengde.
- [ ] Den kan ikke knytte sammen to noder det allerede finnes en sti mellom i A.
- [ ] Det må finnes en oppdeling av nodene i to mengder, slik at kanten knytter sammen to noder som ikke er i samme mengde og har størst vekt blant alle slike kanter. Det er ingen begrensing på om kantene i A knytter sammen to noder i hver sin mengde.
- [ ] Den må knytte sammen to noder det allerede finnes en sti mellom i A.
- [ ] Det må finnes en oppdeling av nodene i to mengder, slik at kanten knytter sammen to noder som ikke er i samme mengde og har størst vekt blant alle slike kanter. Kantene i A kan ikke knytte sammen to noder som kommer fra hver sin mengde.

Question 9: Høyere utdanning

Høyere utdanning i Norge har de siste årene vært preget av flere fusjoner. Dette kan gjøre det vanskelig å holde orden, så vi trenger derfor din hjelp til å implementere et program som kan holde styr på de nye navnene til institusjonene.

Du skal fullføre klassen `HigherEdSolver`. Den skal inneholde implementasjoner av følgende funksjoner:

- `initialize(self, institutions)` – Kalles én gang ved konstruksjon av et `HigherEdSolver`-objekt, og tar inn en liste som inneholder alle institusjonene som skal håndteres. Her kan du initialisere datastrukturene du trenger.
- `fuse(self, institution1, institution2, new_institution)` – Slår sammen institusjonene med navn `institution1` og `institution2` til en ny institusjon med navn `new_institution`. Alle institusjonene som lå under `institution1` eller `institution2` før sammenslåingen skal nå ligge under den nye institusjonen. `institution1` og `institution2` vil aldri ligge under noen annen institusjon når `fuse` kalles.
- `parent_institution(self, institution)` – Returnerer en tekststreng som representerer institusjonen `institution` ligger under, eller `institution` hvis den ikke ligger under noen annen institusjon. Dersom `institution` ligger under en annen institusjon, som igjen ligger under en annen institusjon, så skal den *øverste* institusjonen returneres.

I alle tilfellene er institusjonene representert ved navn (tekststrenger).

Eksempel:

1. `initialize(["UniA","UniB", "UniC"])`
2. `parent_institution("UniB")` skal returnere `"UniB"` siden institusjonen ikke ligger under en annen institusjon.
3. `fuse("UniA", "UniB", "UniAogB")`
4. `parent_institution("UniA")` skal returnere `"UniAogB"` siden `"UniA"` ligger under denne etter sammenslåingen.
5. `parent_institution("UniAogB")` skal returnere `"UniAogB"` siden denne institusjonen ikke ligger under en annen institusjon.
6. `fuse("UniAogB", "UniC", "UniABC")`
7. `parent_institution("UniB")` skal returnere `"UniABC"` siden `"UniB"` ligger under denne etter sammenslåingene.

Ønsker du teste programmet ditt lokalt, finnes det eksempeltester her.

Hint 1

```
class HigherEdSolver:
    def initialize(self, institutions):
        # Skriv din kode her

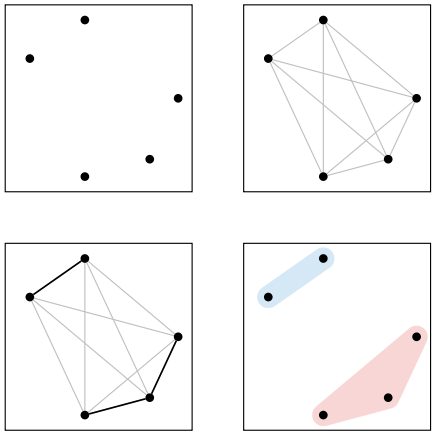
    def fuse(self, institution1, institution2, new_institution):
        # Skriv din kode her

    def parent_institution(self, institution):
        # Skriv din kode her
```

Question 10: Klyngeanalyse

En mulig anvendelse av minimale spenntrær er klyngeanalyse (*clustering*). I klyngeanalyse

ønsker vi å gruppere elementer i k ulike klynger, slik at elementene innad i en klynge er likere hverandre enn elementene utenfor klyngen. Dette kan vi oppnå ved å lage et minimalt spenntre (f.eks. med Kruskals algoritme) basert på en avstandsfunksjon, men stoppe når vi har k trær (altså ikke legge til de siste $k-1$ kantene som trengs for et minimalt spenntre). Dette er ekvivalent med å fjerne de $k-1$ tyngste kantene i et minimalt spenntre. Ved å bruke minimale spenntrær kan vi garantere at vi maksimerer den minste avstanden mellom elementer i forskjellige klynger. Vi maksimerer altså *seperasjonsavstanden* til klyngene. Figuren under viser et eksempel på gruppering av fem elementer i to klynger.



I denne oppgaven skal vi utføre klyngeanalyse på dyr basert på gensekvensene deres. Som avstandsfunksjon skal vi anvende Hamming-avstanden mellom gensekvensene til dyrene. Hamming-avstanden til to strenger av lik lengde er antall posisjoner det står ulike symboler (bokstaver) på i de to strengene. F.eks. vil `"AAB"` og `"ABB"` ha en Hamming-avstand på 1, mens den for `"BAB"` og `"ABA"` vil være 3.

I denne oppgaven skal du fullføre implementasjonen av `find_animal_groups(animals, k)` (du kan også lage din egen implementasjon hvis du ønsker). Her er `k` antall klynger og `animals` en liste av dyr på formen `(navn, gensekvens)`. Funksjonen skal returnere en liste bestående av k lister, hvor hver liste representerer en klynge og består av navnene til dyrene som finnes i denne klyngen. Inndelingen av dyrene i klynger må maksimere seperasjonsavstanden. Er det flere mulige inndelinger som maksimerer seperasjonsavstanden, kan du returnere hvilken som helst av disse. Implementasjonen av `find_animal_groups` gjør følgende:

1. Lager en liste med Hamming-avstanden mellom hvert par av dyr, med indekser i og j, på formen `(i, j, avstand)`.
2. Lager klynger basert på listen produsert i steg 1.
3. Omgjør klyngene produsert i steg 2 til å bruke navn og ikke indekser.

Det som mangler i implementasjonen er utregning av Hamming-avstander (i `hamming_distance`) og hele steg 2. Steg 2 kan implementeres i `find_clusters`, som tar inn listen fra steg 1 (`E`), antall dyr (`n`) og antall klynger (`k`). Funksjonen skal returnere en liste på samme format som `find_animal_groups`, men bruke indekser i stedet for navn.

Eksempel: Vi skal gruppere dyrene `Ugle`, `Mus` og `Rotte` som vi lar ha gensekvenser `AGAG`, `CTTC` og `CGTC`. Vi ønsker å lage to klynger. I steg 1 produseres listen `[(0, 1, 4), (0, 2, 3), (1, 2, 1)]`, siden Hamming-avstanden til `AGAG` og `CTTC` er 4, mens den er 3 for `AGAG` og `CGTC` og 1 for `CTTC` og `CGTC`. Funksjonen i steg 2 tar inn denne listen som `E`, samt verdiene `n = 3, k = 2`. Siden gensekvensene til mus og rotter har lavest Hamming-avstand, havner disse dyrene i samme klynge, og funksjonen returnerer listen `[[0], [1, 2]]`. I steg 3 gjøres denne listen om til `[["Ugle"], ["Mus", "Rotte"]]`.

Ønsker du teste programmet ditt lokalt, finnes det eksempeltester her.

Hint 1 Hint 2

```python
def hamming_distance(s1, s2):
    # Skriv koden din her


def find_clusters(E, n, k):
    """
    Finner k klynger ved hjelp av kantene i E. Kantene i E er på
    formatet (i, j, avstand), hvor i og j er indeksen til noden
    (dyret) kanten knytter sammen og avstand er Hamming-avstanden
    mellom gensekvensene til dyrene. Funksjonen returnerer en liste
    av k lister, hvor hver av de indre listene representerer en
    klynge og består av indeksene til nodene (dyrene). F.eks. om vi
    har tre dyr som skal i to klynger, hvor dyr 0 og 2 ender i samme
    klynge, returnerer funksjonen [[0, 2], [1]].

    :param E: Kanter i grafen på formatet (i, j, avstand). i og j er
              indeksen til dyrene som kanten går mellom.
    :param n: Antall noder
    :param k: Antall klynger som ønskes
    :return:  En liste av k lister
    """
    # Skriv koden din her


def find_animal_groups(animals, k):
    # Lager kanter basert på Hamming-avstand
    E = []
    for i in range(len(animals)):
        for j in range(i + 1, len(animals)):
            E.append((i, j, hamming_distance(animals[i][1], animals[j][1])))

    # Finner klynger
    clusters = find_clusters(E, len(animals), k)

    # Endre fra klynger basert på indekser til klynger basert på dyrenavn
    animal_clusters = [[animals[i][0] for i in cluster] for cluster in clusters]
    return animal_clusters
```

Question 11: Prim

Vi lar Q være en maks-prioritetskø i $\mathrm{MST\text{-}PRIM}$ og endrer prosedyren slik:

```
MST-PRIM'(G, w, r)
 1  for each vertex u ∈ G.V
 2      u.key = −∞
 3      u.π = NIL
 4  r.key = 0
 5  Q = ∅
 6  for each vertex u ∈ G.V
 7      INSERT(Q, u)
 8  while Q ≠ ∅
 9      u = EXTRACT-MAX(Q)
10      for each vertex v ∈ G.Adj[u]
11          if v ∈ Q and w(u, v) > v.key
12              v.π = u
13              v.key = w(u, v)
14              INCREASE-KEY(Q, v, w(u, v))
```

Hva vil $\mathrm{MST\text{-}PRIM'}$ ende opp med? Anta at G er en sammenhengende graf.

- ○ Alltid et minimalt spenntre
- ○ Ingen av de andre alternativene
- ○ Alltid et maksimalt spenntre

Question 12: Kruskal

Hvis vi sorterer kantene i synkende rekkefølge i stedet for stigende i $\mathrm{MST\text{-}KRUSKAL}$, hva vil vi ende opp med? Anta at input er en sammenhengende graf.

- ○ Alltid et minimalt spenntre
- ○ Ingen av de andre alternativene
- ○ Alltid et maksimalt spenntre

Question 13: Graf med én sykel

Denne oppgaven teller ikke som en av oppgavene som kreves for å få øvingen godkjent. Den er kun ment som trening. Grunnet oppgavens form, rettes den ikke automatisk.

Du har en sammenhengende graf G med $|\mathrm{E}| \geqslant |\mathrm{V}|$ og kantvekter $w(e) > 0$. Du ønsker å fjerne kanter slik at du står igjen med en sammenhengende graf med minst én sykel, og med så lav sum av kantvekter som mulig. Hvordan kan du endre $\mathrm{MST\text{-}KRUSKAL}$ for å løse dette problemet?

Question 14: Beslutningsprosedyrer

Denne oppgaven er basert på en eksamensoppgave.

Programvarer for å løse såkalte SMT-problemer brukes ofte til å sjekke at programmer fungerer som de skal, og for å hindre sikkerhetshull. SMT-programvare brukes blant annet mye i symbolsk evaluering. En vanlig komponent i SMT-programvare er en beslutningsprosedyre for en bestemt teori, som har som oppgave å sjekke kandidatløsninger.

Her skal du implementere en forenklet versjon av en beslutningsprosedyre. Du får en mengde med variabler x_i, en mengde med ligninger $x_i = x_j$ og en mendge med ulikheter $x_i > x_j$ eller $x_i < x_j$, og skal sjekke om det er mulig å gi tallverdier til alle variablene slik at alle ligningene og alle ulikhetene overholdes.

Mer bestemt skal du implementere `check(variables, constraints)`. Her er `variables` en liste med tekststrenger som hver representerer en variabel. Parameteren `constraints` er en liste med tupler på formen `(a, comp, b)`, der `a` og `b` er tekststrenger fra `variables` og `comp` kan være `"="`, `"<"` eller `">"`, og representerer henholdsvis at $a = b$, $a < b$ eller $a > b$. Funksjonen skal returnere `True` hvis det er mulig å gi variablene i `variables` tallverdier som overholder restriksjonene i `constraints` og `False` ellers.

Ønsker du teste programmet ditt lokalt, finnes det eksempeltester her. Trenger du flere instanser å teste på lokalt, finnes det et større testsett som kan lastes ned.

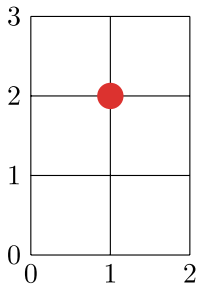
Hint 1 | Hint 2 | Hint 3 | Hint 4 | Hint 5

```
1  def check(variables, constraints):
2      # Skriv din kode her
```

Question 15: Overføringsnett

Denne oppgaven er vanskelig. Fortvil ikke om du ikke får den til.

I en sentrum av en storby er det behov for å bygge ut strømnettet. Det er overføringskapasiteten fra kraftnettet til nettstasjonene (såkalte trafokiosker) som skal utbedres. Storbyen er formet som et regulert rutenett med $m \times n$ veier. Veiene som går horisontalt er nummerert fra 0 til $m - 1$, og veiene som går vertikalt er nummerert fra 0 til $n - 1$. En posisjon i veinettet er gitt av (i, j) som representerer krysset mellom vei i som går horisontalt og vei j som går vertikalt. Et eksempel er vist på figuren nedenfor med $m = 4$ og $n = 3$:



Her er hver kant en vei, og posisjonen $(2, 1)$ er merket med en rød sirkel.

Du skal implementere funksjonen `power_grid(m, n, substations)`, hvor `m` og `n` er som forklart over. Posisjonene til nettstasjonene er gitt av tupler på formen `(i, j)` som sier at det er en nettstasjon i krysset gitt av posisjonen (i, j). Alle nettstasjonene ligger i ulike kryss.

Det skal legges kraftledninger i bakken. Alle kraftledningene må gå langs veiene, siden man ellers må grave opp en bygning, noe som naturligvis er for dyrt. Disse nye kraftledingene skal koble sammen alle nettstasjonene (potensielt via andre nettstasjoner). Det er ønskelig å gjøre dette så billig som mulig ved å gjøre totallengden til ledningene så liten som mulig, målt i antall veistrekninger. En veistrekning her går fra et kryss til et nabokryss (altså fra posisjon (i, j) til $(i + 1, j)$, $(i - 1, j)$, $(i, j + 1)$ eller $(i, j - 1)$). En nettstasjon kan være koblet til flere kraftledninger, og en veistrekning kan ha flere kraftledninger, men en kraftledning kan ikke forgrene seg.

Funksjonen skal returnere totallengden av kraftledningene som må legges, der lengden måles i antall veistrekninger. Kjøretiden til funksjonen bør være $\mathrm{O}(mn \lg(mn))$.

Ønsker du teste programmet ditt lokalt, finnes det eksempeltester her.

Hint 1

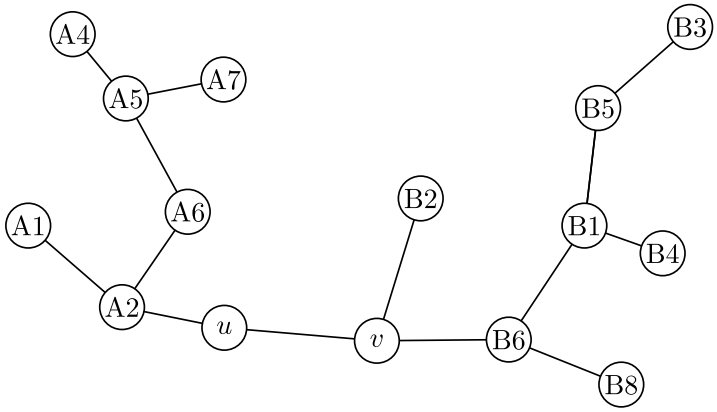
Denne oppgaven teller mot poengtavlen til denne øvingen. For å kunne havne på poengtavlen må du løse oppgaven og sette variabelen `highscore` til `True` i koden din.

Rangeringen din er baseret på hvor lang tid funksjonen din bruker på å løse testene.

```
1  def power_grid(m, n, substations):
2      # Skriv din kode her
3
4  # Sett 'highscore' til True hvis du vil vises på poengtavlen.
5  # For mer info se
   'https://algdat.idi.ntnu.no/ovinger.html#poengtavle'
6  # Merk: Kjøring for poengtavlen tar betraktelig lengre tid.
```

Question 16: Spenntre med splitt-og-hersk

Du ser på et minimalt spenntre lignende det illustrert under:



Du fjerner så kanten mellom u og v. Nodene som fortsatt kan nås fra henholdsvis u og v kaller vi V_u og V_v. Kantene som er igjen representerer spenntrær for V_u og V_v, men er disse spenntrærne nødvendigvis minimale?

- ○ Nei
- ○ Ja

Question 17: Spenntre med splitt-og-hersk

Denne oppgaven teller ikke som en av oppgavene som kreves for å få øvingen godkjent. Den er kun ment som trening. Grunnet oppgavens form, rettes den ikke automatisk.

Kom med et bevis eller moteksempel for påstanden i oppgaven over om at spenntrærne for V_u og V_v er minimale.

Question 18: Spenntre med splitt-og-hersk

Du antar at riktig svar er «ja» i oppgavene over, og kommer dermed til følgende «åpenbaring»: Et minimalt spenntre består av minimale spenntrær på undermengder av noder i grafen. Dette ønsker du å bruke til å lage en splitt-og-hersk-algoritme for å finne minimale spenntrær. Du lager algoritmen under:

```
MST-D&C(G, V, w)
 1  if |V| = 1
 2       return ∅
 3  divide V into two sets V_1 and V_2 as evenly as possible
 4  T_1 = MST-D&C(G, V_1, w)
 5  T_2 = MST-D&C(G, V_2, w)
 6  min-value = ∞
 7  for each vertex u ∈ V_1
 8       for each vertex v ∈ G.adj[u]
 9            if v ∈ V_2 and w(u, v) < min-value
10                 min-edge = (u, v)
11                 min-value = w(u, v)
12  return T_1 ∪ T_2 ∪ {min-edge}
```

Vil algoritmen alltid finne minimalt spenntre dersom input er en sammenhengende graf?

- ○ Nei
- ○ Ja

Question 19: Spenntre med splitt-og-hersk

Denne oppgaven teller ikke som en av oppgavene som kreves for å få øvingen godkjent. Den er kun ment som trening. Grunnet oppgavens form, rettes den ikke automatisk.

Kom med et bevis for, eller moteksempel mot, at algoritmen i oppgaven over gir riktig svar dersom input er en sammenhengende graf.

Hva er kjøretiden til MST-D&C? Gjør rimelige antagelser om hvilke datastrukturer som brukes, og se gjerne også på hvordan ulike datastrukturer påvirker kjøretiden til algoritmen.

Question 20: Egen MST-algoritme

Denne oppgaven teller ikke som en av oppgavene som kreves for å få øvingen godkjent. Den er kun ment som trening. Grunnet oppgavens form, rettes den ikke automatisk.

Ta utgangspunkt i GENERIC-MST og konstruer din egen algoritme for å finne minimale spenntrær. Det er ingen krav til kjøretiden til algoritmen.

Hint 1

Question 21: Anbefalte ekstraoppgaver

Denne oppgaven teller ikke som en av oppgavene som kreves for å få øvingen godkjent. Den er kun ment som trening. Grunnet oppgavens form, rettes den ikke automatisk.

Her følger en liste over anbefalte ekstraoppgaver, som kan være nyttig å se på om du ønsker litt ekstra trening eller som repetisjon mot eksamen. Oppgavene er hentet fra gamle eksamensoppgaver og *Introduction to Algorithms* av Cormen mfl. (CLRS), og er sortert basert på kilde.

- 2024H - 1, 12
- 2024S - 2
- 2023H - 5, 10
- 2023S - 2
- 2021H - 3
- 2020H - 2
- 2020S - 2
- 2019H - 7, 15
- 2019S - 6, 15 (Kan være lurt å se på denne før oppgaven «SMT-solver» i denne øvingen)
- 2018H - 1
- 2017H - 1f)
- 2017S - 6
- 2016H - 14
- 2016S - 17, 20
- 2014H - 1d)
- 2014S - 7, 14
- **CLRS** - 19.1-3
- **CLRS** - 19.3-3, 19.3-4, 19.3-5
- **CLRS** - 19-1, 19-3
- **CLRS** - 21.1-2, 21.1-3, 21.1-4, 21.1-5, 21.1-6, 21.1-8, 21.1-11
- **CLRS** - 21.2-1, 21.2-3, 21.2-4, 21.2-7, 21.2-8
- **CLRS** - 21-1

Submit

Running INGInious v.0.9.2.dev240+gb8a04c56.d20260915
© 2014-2026 Université catholique de Louvain.
INGInious is distributed under AGPL license